Architectural Engineering and Implementation Framework for a High-Performance Toy Browser Engine in C++

The construction of a web browser engine, even one serving as a simplified imitation of modern standards, constitutes a sophisticated exercise in systems programming and software architecture. This report delineates the technical roadmap, underlying mechanisms, and optimization strategies required to develop a functional browser engine utilizing the JSML specification and an embedded Lua runtime environment within a strictly defined seven-day development cycle.[1] The project necessitates a deep integration of document parsing, layout algorithms, concurrent execution models, and hardware-accelerated graphics pipelines to meet the performance benchmarks typical of modern browser engineering.[1]

Theoretical Foundation and the JSML Document Specification

The structural core of the engine relies on JSML, a specialized language that conflates structural markup and stylistic declarations into a unified JSON-like hierarchy.[1] Unlike the traditional separation of concerns found in the HTML/CSS paradigm, JSML facilitates a more direct mapping between the document tree and the rendering instructions, significantly reducing the overhead associated with the CSS cascade and inheritance models.[2]

Document Object Model and Parsing Challenges

The initial stage of the browser pipeline involves the ingestion of raw JSML data and its transformation into an internal Document Object Model (DOM). This tree structure serves as the authoritative state of the webpage, where each node represents either a container or a terminal content element such as text or images.[3]

JSML Element Type	Functional Role	Styling Attributes
flexv	Vertical container for children	spacing, bgcolor
flexh	Horizontal container for children	spacing, bgcolor
text	Terminal node for string rendering	content, fontsize, color
image	Terminal node for bitmap rendering	url, alttext

The parsing process must account for several structural nuances. Because JSML permits C-style comments, standard JSON parsers that strictly adhere to the ECMA-404 standard will encounter fatal errors.[1] Consequently, the implementation must utilize a parser like nlohmann::json or JsonCpp, which provide configurations to treat comments as ignorable whitespace.[4] This decision is critical for maintaining compatibility with user-authored files that utilize comments for documentation or benchmarking directives.[1]

Identification of Technical Ambiguities in JSML

A critical component of the engineering roadmap is the identification and resolution of ambiguities within the problem statement to prevent architectural regressions. The following table summarizes the identified inconsistencies and their proposed resolutions.

Ambiguity	Source Location	Engineering Implication
Syntax Inconsistency	Page 1, Page 2	The example shows properties like bgcolor placed both inside and outside flex arrays. The parser must be resilient to JSON-like but non-compliant object structures.[1]
Tag Property Type	Page 1, Page 2	The tag attribute is variously represented as an integer and a string. The engine should adopt a variant-based ID system to ensure getElemsByTag functions correctly.[1]
Spacing Definition	Page 2	It is unclear if spacing refers to a sibling gap, a margin, or internal padding. Implementing it as a sibling gap (similar to CSS gap) is most consistent with the flexv/flexh descriptions.[1]
Resource Pathing	Page 1	The script and image paths (e.g., /script.lua) lack a protocol. The engine must implement a URI resolver that handles both local file paths and remote network URLs.[1]
Image Load State	Page 2	The text "load" appears

		abruptly in the image description without context. This likely refers to an asynchronous onload event that the engine must support for resource telemetry.[1]

The Layout Engine: Algorithmic Geometry and Flexbox Principles

Once the DOM is established, the engine must perform a layout pass to calculate the geometric bounds of each element. This process transforms the logical tree into a Layout Tree, where each node is assigned an absolute (x, y) coordinate and a definitive $(width, height)$.[3]

Flexbox Mechanics and Coordinate Systems

The engine implements a subset of the Flexbox algorithm, specifically vertical (flexv) and horizontal (flexh) orientations.[1] The layout calculation is a two-pass recursive process. The first pass, the "measurement pass," determines the intrinsic size of terminal nodes. For text elements, the engine must query the font renderer (e.g., FreeType) to determine the bounding box of the string at a given fontsize.[9] For images, the size is derived from the metadata of the decoded bitmap.[1]

The second pass, the "distribution pass," allocates space within containers. For a flexv container, the primary axis is vertical. The height of the container is the sum of the heights of its children plus the accumulated spacing between them.[11] If a container has fixed dimensions, the engine must calculate "positive" or "negative" free space.[11] If the children exceed the available space, they may overflow or shrink, depending on whether the engine implements the flex-shrink property.[10]

$$Height_{container} = \sum_{i=1}^{n} Height_{child_i} + (n-1) \times Spacing$$

The implementation of flex-grow allows elements to expand proportionally to fill available space.[11] This requires calculating a "growth factor" and redistributing the positive free space among eligible children before finalizing their coordinates.[11]

Integration of High-Performance Layout Libraries

To achieve the speed requirements necessary for large-scale benchmarks, the engine can leverage specialized C or C++ layout libraries.[1] Facebook's Yoga library is the industry standard for Flexbox implementations, offering a robust and battle-tested API.[8] Alternatively, the Clay library provides a high-performance, single-header C implementation that focuses on minimizing memory allocations and maximizing traversal speed, making it an ideal candidate for a toy browser engine where binary size and execution speed are prioritized.[6]

Scripting Runtime and the Snapshot-Commit Model

The browser's interactive capabilities are facilitated by an embedded Lua runtime.[1] Lua is chosen for its lightweight nature, ease of integration with C++, and performance characteristics that rival or exceed traditional JavaScript engines in specific embedded contexts.[16]

The Core Philosophy of the Update Cycle

The engine utilizes a "Snapshot and Commit" model for DOM manipulation.[1] Unlike traditional browsers where the DOM is live and potentially volatile, this engine isolates the scripting environment from the rendering state. When a Lua script requests an element via browser.getElem(id), the engine generates a serialized snapshot of the element's current state as a Lua table.[1] Modifications to this table by the script do not immediately affect the UI; instead, the script must explicitly call browser.updateElem(id, table) to "commit" the changes back to the engine.[1]

This model provides several architectural advantages:

1. **Concurrency Safety**: It prevents race conditions where a script might attempt to modify a node while the renderer is traversing the tree for a paint operation.[20]
2. **Batching Potential**: Multiple changes can be made to a Lua table before a single commit call, allowing the engine to minimize the number of layout reflows.[2]
3. **Atomic Consistency**: The UI only transitions to a new state once the commit call is processed, ensuring the user never sees an inconsistent or partially updated frame.[23]

Implementation of the Lua API

The C++ engine must export a suite of functions to the Lua state to enable the required functionality.[1] This integration is best handled through a modern binding library such as sol2, which leverages template metaprogramming to provide high-performance type conversion and function mapping.[17]

Lua Function	C++ Backend Implementation	Purpose
getElemsByTag(tag)	Depth-first search of the DOM tree	Retrieves a list of unique integer IDs for elements.[1]
getElem(id)	Serializes C++ object to Lua table	Provides a mutable snapshot of the element's properties.[1]
updateElem(id, table)	Deserializes Lua table to C++ object	Commits state changes and marks the Layout Tree as "dirty".[1]
addClickHandler(id, cb)	Stores a Lua function reference	Registers an asynchronous callback for user input events.[1]
fetch(url, callback)	Asynchronous networking via	Initiates non-blocking network

	libcurl	transfers with a Lua callback.[1]
getElemRect(id)	Queries the last Layout Tree state	Provides geometric bounds for layout-aware scripting.[1]

Concurrency and Multithreading Architecture

To meet the benchmark requirements for speed, the browser must utilize a sophisticated multithreading model.[1] A single-threaded approach would lead to frame stuttering whenever a Lua script performs intensive calculations or a network request blocks the main loop.[21]

The Triple-Threaded Execution Model

A high-performance browser engine typically divides its workload across three specialized threads.[19] This separation ensures that the user interface remains responsive regardless of the complexity of the page content or scripting logic.[20]

1. **The Main (UI) Thread**: This thread manages the operating system window, polls for input events (mouse and keyboard), and handles the final rendering calls to the GPU.[19] Modern graphics APIs like OpenGL and Vulkan require the graphics context and windowing events to remain on the thread that initialized them to avoid "context explosions" or driver-level deadlocks.[19]
2. **The Logic and Layout Thread**: This thread executes the Lua runtime and performs layout re-calculations.[19] When a Lua script is triggered by a click event or a timer, it runs on this thread. Once the script finishes and potentially calls updateElem, the layout engine performs a reflow and generates a new "Render Tree"—a simplified list of draw commands (rectangles, text, images).[19]
3. **The IO and Networking Thread**: This thread manages libcurl's multi-interface, performing non-blocking network polling.[26] When data (like a JSON response or an image) arrives, this thread decodes the data and passes a notification to the Logic thread to trigger the appropriate callback or update the DOM.[21]

Synchronization via Triple Buffering

Passing the "Render Tree" from the Logic thread to the Main thread requires a synchronization mechanism that avoids long-duration locks.[21] Triple buffering is the optimal solution for this "producer-consumer" problem.[24]

The engine maintains three distinct copies of the render data. The Logic thread (producer) writes to the "back" buffer.[33] Once a new layout is complete, it publishes this buffer to the "spare" buffer.[33] The Main thread (consumer) reads from the "front" buffer to draw the screen.[33] Between frames, the Main thread checks the "spare" buffer for updates; if new data is present, it swaps the "front" and "spare" buffers.[24] This swapping operation is a simple pointer flip, which can be performed atomically in $O(1)$ time, ensuring that neither thread ever blocks the other.[32]

Buffer State	Owner Thread	Current Activity
Front Buffer	Main (UI)	Being used for active GPU draw calls.
Spare Buffer	Shared	Holds the most recent completed Layout Tree.
Back Buffer	Logic	Being filled with new layout/render commands.

High-Performance Rendering and GPU Utilization

The "millions of elements" benchmark requires a rendering strategy that bypasses the limitations of traditional CPU-bound drawing.[1] The engine must offload the heavy lifting of pixel manipulation to the Graphics Processing Unit (GPU).[35]

Batching and Hardware Acceleration

The primary bottleneck in high-performance 2D rendering is the overhead of "draw calls"—the commands sent from the CPU to the GPU.[37] If the browser makes a separate draw call for every flexv rectangle and text node, the CPU will spend more time managing driver state than the GPU spends drawing pixels.[37]

To optimize this, the engine must implement **Render Batching**.[37] All elements that share the same rendering state (e.g., all rectangles of the same color or all text of the same font) are grouped into a single vertex buffer.[37] This buffer is sent to the GPU in a single transaction, allowing the hardware to process thousands of elements in parallel.[35] In an advanced implementation, the engine can utilize "Instanced Rendering," where a single rectangle mesh is stored on the GPU, and the Logic thread only sends an array of "instance data" (position, size, and color) to draw millions of elements at once.[39]

Text Rendering via Texture Atlases

Text is particularly challenging because each glyph is a unique shape.[9] Using FreeType to rasterize every character on every frame is prohibitively slow.[9] Instead, the engine should implement a **Texture Atlas**—a single large image containing every possible character of a font.[9] When drawing text, the engine generates a series of textured quads, where each quad points to a specific region of the atlas.[9] This allows strings of text to be batched together with images, as long as they reside on the same GPU texture.[9]

Advanced GPU Suggestions: Vulkan and Compute Shaders

For "maximum overkill" in performance, the engine could utilize the Vulkan API.[41] Unlike OpenGL, which hides many hardware details, Vulkan provides explicit control over memory

allocation and command submission.[41] A "Snapshot Rendering" approach can be used, where a pre-recorded sequence of draw commands is reused if the DOM hasn't changed, reducing CPU overhead to almost zero.[44]

Furthermore, the layout engine itself could be moved to the GPU using **Compute Shaders**.[35] While the Flexbox algorithm is recursive, many of its properties (like percentage-based width calculations) can be parallelized.[35] A compute-centric engine like Vello or WebGPU's compute pipeline could handle both layout and rendering, offering a future-proof architecture for massive document trees.[35]

Networking and Asynchronous Resource Loading

The engine must support loading JSML pages, Lua scripts, and images over the network.[1] This requires a robust networking stack that does not block the browser's interactivity.[2]

The libcurl Multi-Interface

libcurl is the most reliable library for cross-platform network transfers in C++.[46] The engine should utilize libcurl's multi interface, which is designed specifically for asynchronous, parallel transfers.[26]

The networking workflow is as follows:

1. **Request Initiation**: When a JSML page requires an image or a Lua script, the engine creates a "curl easy handle" and adds it to a "curl multi handle".[26]
2. **The Multi-Loop**: On every frame (or in a dedicated IO thread), the engine calls curl_multi_perform.[30] This function performs a "slice" of the network work, such as reading a few bytes from a socket.[30]
3. **Callback Execution**: When a transfer is complete, libcurl notifies the engine.[26] If the request came from a Lua fetch call, the engine serializes the response and executes the associated Lua callback.[1] If it was an image for the DOM, the engine initiates a texture upload to the GPU.[2]

URI Resolution and Security

The engine must resolve relative paths correctly.[1] If the browser is currently at http://example.com/index.jsml, then a request for /cat.png must be resolved to http://example.com/cat.png.[1] This requires a dedicated URI parsing module that adheres to RFC 3986. For local file access, the engine should implement a basic sandbox to ensure that JSML pages cannot access arbitrary files on the user's Linux system, restricting access to the directory containing the original JSML file or a designated "resources" folder.[16]

Build Systems and Development Infrastructure

The requirement for the application to run on Linux and ideally be cross-platform dictates the use of a standard, flexible build system.[1] **CMake** is the preeminent choice for modern C++

development.[50]

CMake Modularization

A modular CMake structure allows for the clean separation of the engine's components.[50] This is essential for a project of this complexity, where the renderer, the scripting runtime, and the networking stack should be developed as independent libraries or modules.[53]

CMake

```
cmake_minimum_required(VERSION 3.15)
project(ToyBrowser)

set(CMAKE_CXX_STANDARD 17)

# Search for system dependencies
find_package(SDL2 REQUIRED)
find_package(Lua REQUIRED)
find_package(CURL REQUIRED)

# Define the engine library
add_library(BrowserEngine
  src/dom.cpp
  src/layout.cpp
  src/renderer.cpp
  src/scripting.cpp
)
target_link_libraries(BrowserEngine SDL2::SDL2 Lua::Lua CURL::libcurl)

# Define the final executable
add_executable(toy_browser main.cpp)
target_link_libraries(toy_browser BrowserEngine)
```

Performance-Oriented Build Configurations

For the benchmarking phase, the build system should be configured to generate highly optimized binaries.[25] Using the -O3 flag is standard, but developers should also consider **Link Time Optimization (LTO)**, which allows the compiler to perform optimizations across different source files, potentially inlining critical layout functions directly into the main loop.[25] On Linux, the engine should be linked against a high-performance allocator like jemalloc or mimalloc, as the frequent creation and destruction of Lua tables and DOM nodes can otherwise lead to

memory fragmentation and allocation bottlenecks.[25]

The Seven-Day Implementation Roadmap

Building a browser in a week is an aggressive target that requires a highly structured workflow. Each day focuses on a critical pillar of the engine's architecture.[1]

Day 1: Document Model and Parsing

The primary objective is the creation of the DOM structure and the JSML parser. The parser must handle the inconsistent JSON-like syntax and ignore comments.[1] By the end of the day, the engine should be able to load a JSML file from disk and represent it as a tree of C++ objects in memory.[3]

Day 2: The Flexbox Layout Engine

Implement the vertical and horizontal layout distribution. This includes the recursive measurement of text (using a stub for the font renderer) and the allocation of space based on the spacing property.[1] Test this with nested flex containers to ensure the math for parent-child sizing is correct.[10]

Day 3: GPU Renderer and Windowing

Initialize SDL and implement the basic rendering pipeline.[42] Create a system to draw colored rectangles for flex containers and implement the FreeType-based text renderer.[9] By the end of Day 3, the browser should be able to render a static, local JSML page with correct colors and text positions.[37]

Day 4: Embedded Scripting Runtime

Integrate the Lua interpreter using sol2.[17] Implement the browser.getElem and browser.updateElem functions.[1] Create an event system that maps mouse clicks in the window to node IDs in the Layout Tree, triggering the onclick Lua function.[1]

Day 5: Networking and Async Resource Loading

Integrate libcurl and implement the asynchronous fetch API.[26] Develop the image loading pipeline, where images are fetched in the background and uploaded to the GPU as textures once complete.[2] Implement relative path resolution for both scripts and images.[1]

Day 6: Concurrency and Threading Optimization

Move the Lua runtime and the layout calculations to a background thread.[19] Implement the triple-buffering system to pass the Render Tree to the Main thread.[24] Profile the engine to identify performance hot-spots, such as excessive string copies or redundant layout passes.[13]

Day 7: UI Features and Benchmarking

Implement the tab bar and the URL input box using the browser's own rendering primitives.[1] Conduct intensive benchmarking with pages containing thousands of elements.[1] Fine-tune the rendering batching and Lua interop to maximize the frames per second (FPS) and minimize input lag.[32]

Suggestions for Future Enhancement

To transcend the "toy" designation and approach professional standards, the engine could incorporate several advanced architectural improvements.[3]

Hardware-Accelerated Layout via WebGPU

As browser elements become more numerous, the CPU-bound nature of the Flexbox algorithm becomes a bottleneck.[1] Utilizing WebGPU—a modern API that provides low-level access to the GPU for both graphics and general-purpose computation—could allow the engine to perform layout calculations in parallel.[35] A compute shader could calculate the bounding boxes of a million elements in a fraction of the time required by a single CPU thread.[35]

Just-In-Time (JIT) Compilation for Scripting

While Lua is fast, the standard PUC-Rio Lua interpreter is an interpreted bytecode VM.[16] Switching to **LuaJIT** would provide a significant speed boost for complex scripting logic.[16] LuaJIT includes a highly optimized tracer JIT compiler that can execute Lua code at speeds approaching C, which would be a decisive advantage in benchmarks involving intensive DOM manipulation.[16]

Incremental Re-flow and Dirty Marking

Professional engines like Blink and Gecko do not re-calculate the entire layout on every change.[2] Implementing a "dirty marking" system, where only the affected sub-tree is re-calculated, would drastically reduce the computational load.[22] If a Lua script changes the color of a leaf node, the layout remains unchanged; if it changes the fontsize, only that node and its parents need to be reflowed.[22]

Sub-pixel Text Rendering and Advanced Compositing

Text quality can be improved through sub-pixel anti-aliasing, which utilizes the physical structure of LCD pixels to increase the perceived resolution of fonts.[58] Furthermore, a sophisticated compositor could handle transparency and complex "flex-wrap" scenarios by treating each major UI component as a separate GPU layer, allowing for smooth animations even when the underlying document is massive.[13]

Conclusion: Synthesized Roadmap for Success

The development of the toy browser engine requires a harmonious balance between structural correctness and execution speed. By strictly adhering to the "Snapshot and Commit" model, the engine ensures a stable environment for Lua scripting while maintaining the integrity of the rendering pipeline.[1] The transition to a multi-threaded architecture with triple-buffered rendering is the most critical step for meeting high-performance benchmarks, as it effectively removes the scripting runtime from the critical path of the frame loop.[19]

The final submission must be a cohesive Linux application that demonstrates not only the correct rendering of JSML but also the responsiveness of its asynchronous networking and the speed of its layout calculations.[1] Utilizing modern C++ standards, robust build systems like CMake, and industry-standard libraries like SDL2 and libcurl provides the necessary foundation to deliver a professional-grade browser engine within the ambitious seven-day timeframe.[48]

Success in this project is defined by the engine's ability to handle the "millions of elements" stress test, a feat achievable only through aggressive batching, clever concurrency, and the judicious use of GPU acceleration.[1]

Works cited

1. Browser.pdf
2. Show HN: I built a small browser engine from scratch in C++ - Hacker News, accessed March 30, 2026, https://news.ycombinator.com/item?id=46795540
3. Let's build a browser engine! Part 1: Getting started, accessed March 30, 2026, https://limpet.net/mbrubeck/2014/08/08/toy-layout-engine-1.html
4. Comments - JSON for Modern C++, accessed March 30, 2026, https://json.nlohmann.me/features/comments/
5. GitHub - open-source-parsers/jsoncpp: A C++ library for interacting with JSON., accessed March 30, 2026, https://github.com/open-source-parsers/jsoncpp
6. clay/README.md at main · nicbarker/clay · GitHub, accessed March 30, 2026, https://github.com/nicbarker/clay/blob/main/README.md
7. AlexsanderDamaceno/Athenas-Browser-: Web Browser Engine From Scratch in C++, accessed March 30, 2026, https://github.com/AlexsanderDamaceno/Athenas-Browser-
8. Tutorial for CLSS flexbox library for Go, accessed March 30, 2026, https://blog.kowalczyk.info/article/9/tutorial-for-github.comkjkflex-go-package-implementation-of-css-flexbox-algorithm.html
9. Text Rendering - LearnOpenGL, accessed March 30, 2026, https://learnopengl.com/In-Practice/Text-Rendering
10. The Layout Engine - Unity - Manual, accessed March 30, 2026, https://docs.unity3d.com/2020.3/Documentation/Manual/UIE-LayoutEngine.html
11. Controlling ratios of flex items along the main axis - CSS - MDN Web Docs, accessed March 30, 2026, https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Flexible_box_layout/C

ontrolling_flex_item_ratios
12. A Complete Guide to CSS Flexbox, accessed March 30, 2026, https://css-tricks.com/snippets/css/a-guide-to-flexbox/
13. Layout rework and benchmarks - PanGui, accessed March 30, 2026, https://www.pangui.io/blog/05-layout-rework-and-benchmarks/
14. C Implementation of Flexbox - Kevin Powell - Community - Answer Overflow, accessed March 30, 2026, https://www.answeroverflow.com/m/1346120361330344026
15. GitHub - nicbarker/clay: High performance UI layout library in C., accessed March 30, 2026, https://github.com/nicbarker/clay
16. Using Lua with C++ (and C) - Elias Daler, accessed March 30, 2026, https://edw.is/using-lua-with-cpp/
17. When embedding Lua into your C++ game, do you prefer sol2 or just the lua 5.4 library? Why? : r/gamedev - Reddit, accessed March 30, 2026, https://www.reddit.com/r/gamedev/comments/wk4km4/when_embedding_lua_into_your_c_game_do_you_prefer/
18. Integrating Lua in C++ - GeeksforGeeks, accessed March 30, 2026, https://www.geeksforgeeks.org/cpp/integrating-lua-in-cpp/
19. The Poor Man's Threading Architecture · Evan Todd, accessed March 30, 2026, https://etodd.io/2016/01/12/poor-mans-threading-architecture/
20. C++ Multithreading, the simple way | by Josh Weinstein | CodeX - Medium, accessed March 30, 2026, https://medium.com/codex/c-multithreading-the-simple-way-95aa1f7304a2
21. Pro's/Con's of separating game logic and render threads, accessed March 30, 2026, https://gamedev.stackexchange.com/questions/72193/pros-cons-of-separating-game-logic-and-render-threads
22. An Optimal Browser Design for GPU Utilisation | Ekioh, accessed March 30, 2026, https://www.ekioh.com/wp-content/uploads/An-Optimal-Browser-Design-for-GPU-Utilisation.pdf
23. Triple Buffering as a Concurrency Mechanism - GitHub, accessed March 30, 2026, https://github.com/remis-thoughts/blog/blob/master/triple-buffering/src/main/md/triple-buffering.md
24. How I Learned to Stop Worrying and Love Juggling C++ Atomics | brilliantsugar, accessed March 30, 2026, https://brilliantsugar.github.io/posts/how-i-learned-to-stop-worrying-and-love-juggling-c++-atomics/
25. Why sol2 is slower than LuaBridge3? - c++ - Stack Overflow, accessed March 30, 2026, https://stackoverflow.com/questions/71161432/why-sol2-is-slower-than-luabridge3
26. libcurl example - multi-app.c, accessed March 30, 2026, https://curl.se/libcurl/c/multi-app.html
27. My attempt at a multithreaded logic/rendering pipeline. (More info in comments) -

Reddit, accessed March 30, 2026, https://www.reddit.com/r/gamedev/comments/afa6ym/my_attempt_at_a_multithreaded_logicrendering/
28. c++ - Making a separate thread for rendering made my code slower - Stack Overflow, accessed March 30, 2026, https://stackoverflow.com/questions/68058981/making-a-separate-thread-for-rendering-made-my-code-slower
29. A Deep Dive into C++ Concurrency: from fundamentals to production level - Medium, accessed March 30, 2026, https://medium.com/@ragulnath255/a-deep-dive-into-c-concurrency-from-fundamentals-to-production-level-55237231e00a
30. How to do curl_multi_perform() asynchronously in C++? - Stack Overflow, accessed March 30, 2026, https://stackoverflow.com/questions/24288513/how-to-do-curl-multi-perform-asynchronously-in-c
31. libcurl : Handle multiple asynchronous requests in C++ - Stack Overflow, accessed March 30, 2026, https://stackoverflow.com/questions/59778282/libcurl-handle-multiple-asynchronous-requests-in-c
32. Multithreading Problems In Game Design - Erik McClure, accessed March 30, 2026, https://erikmcclure.com/blog/multithreading-problems-in-game-design/
33. Triple Buffer: Lock-free Concurrency Primitive | by Ng Song Guan - Medium, accessed March 30, 2026, https://medium.com/@sgn00/triple-buffer-lock-free-concurrency-primitive-611848627a1e
34. How to correctly implement triple buffering? - Stack Overflow, accessed March 30, 2026, https://stackoverflow.com/questions/49352853/how-to-correctly-implement-triple-buffering
35. WebGPU API - MDN Web Docs - Mozilla, accessed March 30, 2026, https://developer.mozilla.org/en-US/docs/Web/API/WebGPU_API
36. GPU-accelerated CSS properties - Amit Pal - Medium, accessed March 30, 2026, https://helloamitpal.medium.com/gpu-accelerated-css-properties-de656c1ced97
37. Renderer vs SDL_gpu for text rendering? - SDL Development ..., accessed March 30, 2026, https://discourse.libsdl.org/t/renderer-vs-sdl-gpu-for-text-rendering/31977
38. Faster Draw - Vulkan Guide, accessed March 30, 2026, https://vkguide.dev/docs/new_chapter_5/faster_draw/
39. 2D game rendering a million sprites using using Vulkan 1.3 API - YouTube, accessed March 30, 2026, https://www.youtube.com/watch?v=tgAuimhDKj4
40. Rendering text and images performance help? : r/opengl - Reddit, accessed March 30, 2026, https://www.reddit.com/r/opengl/comments/rw6r53/rendering_text_and_images_performance_help/
41. Using Modern Vulkan in 2025 - Medium, accessed March 30, 2026,

https://medium.com/@allenphilip78/using-modern-vulkan-in-2025-0bac45174304
42. Game Development in C: Raylib or SDL2 (or even other options in C)? - Reddit, accessed March 30, 2026, https://www.reddit.com/r/C_Programming/comments/aos36l/game_development_in_c_raylib_or_sdl2_or_even/
43. Advanced API Performance: Vulkan Clearing and Presenting | NVIDIA Technical Blog, accessed March 30, 2026, https://developer.nvidia.com/blog/advanced-api-performance-vulkan-clearing-and-presenting/
44. WebGPU: Unlocking modern GPU access in the browser | Blog - Chrome for Developers, accessed March 30, 2026, https://developer.chrome.com/blog/webgpu-io2023
45. zhanba/awesome-2d-graphics-rendering - GitHub, accessed March 30, 2026, https://github.com/zhanba/awesome-2d-graphics-rendering
46. libcurl - source code examples, accessed March 30, 2026, https://curl.se/libcurl/c/example.html
47. libcurl - programming tutorial, accessed March 30, 2026, https://curl.se/libcurl/c/libcurl-tutorial.html
48. libcurl - API overview, accessed March 30, 2026, https://curl.se/libcurl/c/libcurl.html
49. Lua-cURL/Lua-cURLv3: Lua binding to libcurl · GitHub - GitHub, accessed March 30, 2026, https://github.com/Lua-cURL/Lua-cURLv3
50. Some CMake Templates (examples). Qt, Boost, OpenCV, C++11, etc 一些栗子 - GitHub, accessed March 30, 2026, https://github.com/district10/cmake-templates
51. CMake for Cross-Platform Compilation | Baeldung on Linux, accessed March 30, 2026, https://www.baeldung.com/linux/cmake-cross-platform-compilation
52. CMake template for C++ library (static/shared & Windows/Linux) project : r/cpp - Reddit, accessed March 30, 2026, https://www.reddit.com/r/cpp/comments/vjiv7a/cmake_template_for_c_library_staticshared/
53. toy - the thin c++ game engine, accessed March 30, 2026, https://hugoam.github.io/toy-io/
54. hugoam/toy: the thin c++ game engine - GitHub, accessed March 30, 2026, https://github.com/hugoam/toy
55. Which JSON library do you recommend for C++? : r/cpp_questions - Reddit, accessed March 30, 2026, https://www.reddit.com/r/cpp_questions/comments/1pn2pmi/which_json_library_do_you_recommend_for_c/
56. raylib vs SDL - A libraries comparison - GitHub Gist, accessed March 30, 2026, https://gist.github.com/raysan5/17392498d40e2cb281f5d09c0a4bf798
57. Using Lua with C++ - Hacker News, accessed March 30, 2026, https://news.ycombinator.com/item?id=34782491
58. Easy Scalable Text Rendering on the GPU | by Evan Wallace | Medium, accessed March 30, 2026,

https://medium.com/@evanwallace/easy-scalable-text-rendering-on-the-gpu-c3f4d782c5ac